

# Mini Paper — 2.3 Algorithms

Time: ~35 minutes · Total: 30 marks · Answer all questions.

Mark your work against the mark scheme at the end; aim for ~1 minute per mark. Show your traces.

## Questions

**Q1 [2] (AO1).** State what is meant by *Big O notation* and explain why constant factors and lower-order terms are ignored.

.....

.....

.....

**Q2 [3] (AO1).** State the worst-case time complexity of each of the following, using Big O notation:

.....

.....

.....

.....

(a) Linear search of an unsorted list of  $n$  items. (b) Binary search of a sorted list of  $n$  items. (c) A naive algorithm whose number of operations doubles each time one extra item is added to the input.

**Q3 [5] (AO2).** A list contains `[6, 2, 5, 3]`. Carry out an **insertion sort**, showing the state of the list after **each** value has been inserted into its correct position. State the time complexity of insertion sort in the **worst case**.

.....

.....

.....

.....

.....

.....

.....

.....

**Q4 [4] (AO2).** Trace a **binary search** for the value **19** in the sorted list below (indices 0–6):

|        |   |   |    |    |    |    |    |
|--------|---|---|----|----|----|----|----|
| Index: | 0 | 1 | 2  | 3  | 4  | 5  | 6  |
| Value: | 3 | 7 | 11 | 15 | 19 | 23 | 27 |

Show the values of `low`, `high` and `mid` at each step, and state how many comparisons of `list[mid]` are needed. State the precondition that must hold for binary search to work.

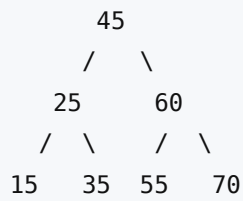
**Q5 [4] (AO1/AO2).** A program uses a nested loop in which an outer loop runs  $n$  times and, for each outer iteration, an inner loop runs up to  $n$  times.

(a) State the time complexity of this algorithm in Big O notation. **[1]**

(b) Justify your answer. [2]

(c) Name one standard sorting algorithm whose average-case complexity is this value. [1]

**Q6 [4] (AO2).** For the binary search tree below, give the **pre-order**, **in-order** and **post-order** traversal output, and state what is special about the in-order output.



---

---

---

---

---

---

---

---

**Q7 [8] (AO3).** Using **Dijkstra's algorithm**, find the shortest distance and the shortest **path** from **A** to **E** in the weighted, undirected graph below. Show your working in a table of tentative distances and the order in which nodes are visited.

Edges (undirected, weighted):

$$A-B = 4, \quad A-D = 1, \quad B-D = 2, \quad B-C = 3, \quad B-E = 6, \quad C-E = 2, \quad D-E = 7$$
[illegible]